

MEMORIA DEL PROYECTO - BD2

Participantes: Bustos Hermoso, Pablo; García Trujillo, Daniel; Gheorghe, Miguel Adrián; Martín Cortés, Francisco Javier

1. DESCRIPCIÓN DEL SISTEMA

El sistema de base de datos desarrollado da soporte integral a la logística y administración de la PAU. El modelo está diseñado para gestionar de forma robusta e integrada un censo masivo de más de 40 * 150 estudiantes, la matriculación en materias específicas, la asignación física de sedes de examen mediante balanceo de aforos, el nombramiento de tribunales (vocales y vigilantes), el control de asistencias en tiempo real y el blindaje de la información mediante capas de auditoría, privacidad y seguridad.

2. DISEÑO RELACIONAL Y RESTRICCIONES SEMÁNTICAS

- **Validaciones (CHECK):** Las columnas de control como `ASISTE` y `ENTREGA` (tabla `ASISTENCIA`), así como los requerimientos de necesidades especiales en la tabla `ANE` (`DESCABEZAR`, `AULA_APARTE`), están estrictamente limitadas por la regla `CHECK (columna IN ('S', 'N'))`. Asimismo, la columna `ALUMNOS_PRESENTES` en `EXAMEN` restringe aforos negativos mediante `CHECK (ALUMNOS_PRESENTES >= 0)`.
- **Unicidad (UNIQUE):**
 - `MATERIA.NOMBRE`: Evita registrar la misma asignatura bajo diferentes códigos.
 - `SEDE.NOMBRE` y `SEDE.DNI_RESPONSABLE`: Garantizan que no existan nombres de sedes duplicados y que un Vocal solo pueda ejercer la jefatura de una única sede simultáneamente.
 - `USUARIO_ORACLE` (en `ESTUDIANTE` y `VOCAL`): Bloquea matemáticamente la duplicidad de cuentas de inicio de sesión en el servidor, admitiendo múltiples valores nulos condicionales durante la precarga masiva del censo estudiantil de los archivos externos.

3. RENDIMIENTO, OPTIMIZACIÓN Y CARGA DE DATOS

3.1. Indexación y Tablespaces

- **Índices Basados en Funciones (IDX_EST_APELLIDOS_UPPER):** Las búsquedas de alumnos por apellidos son recurrentes. Al implementar un índice funcional basado

en `UPPER(APELLIDOS)`, se logran consultas insensibles a mayúsculas/minúsculas instantáneas que omiten los costosos escaneos completos.

- **Índices de Mapa de Bits (Bitmap):** Configurados en columnas de baja cardinalidad (`ASISTE`, `ENTREGA`). Optimizan las funciones complejas como las de agregación (`COUNT`).
- **Aislamiento en Tablespaces:** Las tablas operacionales se almacenan en `TS_PAU`, mientras que sus índices asociados se redirigen físicamente al almacenamiento `TS_INDICES`.

3.2. Vistas Materializadas (`VM_ESTUDIANTES`)

Para proteger el rendimiento, los reportes estadísticos agregados solicitados por el Vicerrectorado se delegan en la `VM_ESTUDIANTES`. Se configuró una política de refresco diario (`REFRESH COMPLETE START WITH SYSDATE NEXT SYSDATE + 1`).

4. LÓGICA DE NEGOCIO (PAQUETES Y TRIGGERS)

Los procesos complejos del sistema se encapsularon en componentes PL/SQL reutilizables:

- **Paquete `PK_ASIGNA`:** La función `F_PLAZAS` calcula el espacio neto disponible cruzando la capacidad de las aulas con los alumnos ya ubicados. El procedimiento `PR_ASIGNA_SEDE` ejecuta un reparto geográfico en dos fases: empareja centros coincidentes por nombre con sedes oficiales y luego recorre el resto de institutos en un cursor ordenado decrecientemente, asignándose de forma equilibrada a la sede con mayor aforo libre en cada iteración de bucle.
- **Paquete `PK_OCUPACION`:** Encapsula funciones de control como `OCUPACION_MAXIMA` (suma alumnos y personal simultáneo en un aula) y `OCUPACION_OK` (validador futuro de límites de habitabilidad).
- **Triggers Operacionales y Contingencia:** El procedimiento `DESPISTE` resuelve incidencias reubicando en cascada los exámenes de alumnos que acuden a sedes erróneas si falta menos de una hora para la prueba. El trigger `TR_MIGRAR_CENTRO` automatiza el desplazamiento en bloque de las asistencias cuando un instituto cambia de sede asignada, y `TR_BORRA_AULA` actúa como frontera impidiendo la eliminación física de un aula si tiene exámenes planificados en un margen crítico de 48 horas.

5. SEGURIDAD, PRIVILEGIOS Y AUDITORÍA

5.1. Control de Privilegios y Gestión con `WITH CHECK OPTION`

Se estructuraron los roles `R_ESTUDIANTE`, `R_VOCAL` y `R_SERVICIO_ACCESO` limitando estrictamente sus permisos sobre las vistas asignadas. Las interfaces del Responsable de Sede se blindaron mediante la cláusula `WITH CHECK OPTION` en las vistas, impidiendo que

un usuario inserte o modifique datos de aulas, exámenes o vigilantes que pertenezcan a una sede ajena a su jurisdicción.

5.2. Blindaje contra SQL Injection mediante DBMS_ASSERT

En los procedimientos de creación de cuentas (PR_CREA_ESTUDIANTE y PR_CREA_VOCAL), se concatenan cadenas para ejecutar sentencias DDL de base de datos, lo que expone al sistema a ataques de inyección de código.

Para blindar el sistema, se integró el uso de la función oficial DBMS_ASSERT.ENQUOTE_NAME. Esta rutina analiza la estructura del identificador construido, verifica que cumpla con los estándares lógicos de nombres de Oracle y lo envuelve entre comillas dobles. De este modo, si un atacante introduce caracteres de escape, espacios o comandos con punto y coma (;), la función lo detecta, anula su ejecución convirtiéndolo en simple texto plano inofensivo y aborta la transacción con un error de seguridad, impidiendo de raíz la inyección.

5.3. Auditoría y VPD

Se implementó un registro apoyado en la tabla AUDIT_ASISTENCIA y el trigger TR_AUDIT_ASISTENCIA, capturando de forma automática el usuario real de la sesión, la operación efectuada (MODIFICACION / BORRADO), la marca de tiempo y los estados previos y nuevos ante cambios en las asistencias. Adicionalmente, mediante DBMS_RLS, se activó una política de Base de Datos Privada Virtual (VPD) que inyecta un predicado invisible a las consultas de los estudiantes, restringiendo el acceso exclusivamente a sus propios datos.

6. METODOLOGÍA DE TRABAJO Y USO DE IA

- **Uso de Inteligencia Artificial:** Se hizo uso de modelos de Inteligencia Artificial (IA) para agilizar el desarrollo. En concreto, la IA facilitó la generación de los extensos bancos de pruebas (tests) y el planteamiento lógico inicial para las rutinas complejas.
- **Garantía de Funcionamiento mediante Bancos de Pruebas Automatizados:** La fiabilidad de todo el script se garantiza mediante el diseño de bloques de pruebas inmediatamente debajo de cada funcionalidad. Estos bloques extraen datos ya cargados, simulando escenarios y validando el éxito o fallo controlado mediante mensajes en la consola DBMS_OUTPUT, cerrando siempre las simulaciones con la instrucción ROLLBACK para dejar todos los datos intactos.
- **Supervisión Humana y Cambios Definitivos:** El equipo mantuvo en todo momento el control del software. Las sugerencias y estructuras planteadas por la IA fueron analizadas, depuradas y corregidas manualmente por los integrantes del grupo. Un claro ejemplo de ello fue en el paquete PK_SEGURIDAD_PAU, donde a la hora de darnos la estructura de los procedimientos, no tuvo en cuenta la defensa a ataques de SQL Injection, el cuál tuvimos que resolver, asegurando que los cambios y decisiones de diseño definitivas fueran tomadas íntegramente por los autores humanos.